Carleton University

Department of Systems and Computer Engineering

SYSC 4907 - Capstone Project

Smart Scheduler for Carleton Engineering Students

Final Report

Group G67

Anas Ayoubi (101226450)

Hakeem Khan (101233039)

Pietro Adamvoski (101238885)

Safiullah Rattar (101245709)

Supervisor: Prof. Jerome Talim

A report submitted in partial fulfillment of the requirements

of SYSC 4907 Engineering Project

Faculty of Engineering

Carleton University

Abstract

The Smart Scheduler is a web application that helps Carleton students plan their course timetables. It parses a student's academic audit, checks prerequisite eligibility, and generates conflict-free schedules from the available course sections. We built it as a full-stack system with a React and TypeScript frontend, an Express and Node.js backend, and a PostgreSQL database holding the course, section, and prerequisite data. This report explains the requirements, design decisions, implementation, and testing work behind the project. It also covers the features we cut, the parts that still feel unfinished, and what we learned over the two terms.

Contents

1 Introduction 1

1.1 Motivation 1

1.2 Project Objective 2

1.3 Report Overview 2

2 The Engineering Project 4

2.1 Health and Safety 4

2.2 Engineering Professionalism 5

2.3 Project Management 5

2.4 Justification of Suitability for Degree Program 6

2.5 Individual Contributions 8

2.5.1 Project Contributions 8

2.5.2 Report Contributions 8

3 Requirements and Design 10

3.1 Original Requirements 10

3.2 System Architecture 13

3.2.1 Architectural Decision: Separated Frontend and Backend 13

3.2.2 High-Level Architecture 14

3.2.3 Design Patterns 15

3.3 Database Design 16

3.3.1 Schema Design Process 16

3.3.2 Final Schema 18

3.3.3 Data Population 20

3.4 Descoped Features and Rationale 20

4 Implementation 22

4.1 Prerequisite Validation Engine 22

4.1.1 Problem Statement 22

4.1.2 Data Structure 22

4.1.3 Algorithm 23

4.1.4 Why the Traversal Was Non-Trivial 24

4.1.5 Grade Comparison 25

4.1.6 Design Decision: Recursive vs. Iterative 25

4.2 Schedule Generation Algorithm 25

4.2.1 Problem Definition 25

4.2.2 Algorithm Selection 26

4.2.3 Implementation 27

4.2.4 Conflict Detection 28

4.2.5 Backtracking Example 28

4.2.6 Deterministic Behavior 29

4.2.7 Performance 30

4.3 Academic Audit Parsing 30

4.3.1 HTML Parsing 30

4.3.2 PDF Parsing 31

4.3.3 Design Decision: `pdftotext` vs. JavaScript Library 33
4.4 Backend API 33
4.5 Frontend Interface 34
4.5.1 Login Page 37
4.5.2 Page 1: Academic History Import 37
4.5.3 Page 2: Course Selection 37
4.5.4 Page 3: Schedule Review 38
4.5.5 Admin Page 39
4.5.6 UI Technology 39
5 Testing and Evaluation 40
5.1 Testing Approach 40
5.1.1 Manual Functional Testing 40
5.1.2 CLI Testing Tool 42
5.2 Requirements Verification 43
5.3 Limitations of Testing Approach 45
6 Reflections 46
6.1 Achievement of Original Objectives 46
6.2 What Changed and Why 47
6.3 Team Dynamics 48
6.4 Hindsight Decisions 48
6.5 Overall Assessment 49
7 References 51

H Appendix A: API Reference 53

H.0.1 Schedule Generation Request Example 53

H.0.2 Schedule Generation Response Example 53

I Appendix B: Database Statistics 55

J Appendix C: Original Project Proposal 56

List of Tables

Table 1 Functional Requirements and Final Status 11

Table 2 Non-Functional Requirements and Final Status 12

Table 3 Database Schema Iterations for Prerequisite Modeling 17

Table 4 Database Schema Summary 19

Table 5 Schedule Generation Performance 30

Table 6 REST API Endpoints 34

Table 7 Functional Requirements Verification Matrix 44

List of Appendices

- Appendix A: API Reference
- Appendix B: Database Statistics
- Appendix C: Original Project Proposal

1 Introduction

1.1 Motivation

Every term, Carleton students go through the same routine: open your academic audit, open the undergraduate calendar, open the timetable, and try to piece together a schedule that actually works. Carleton Central is an enrollment system, not a planning tool. It does not tell you which electives you are still eligible for, and it does not help you work through the larger planning problem of building a timetable that fits your degree progress and your actual life.

Part of the problem is that prerequisite rules at Carleton are not always simple. Some courses require specific combinations of other courses, minimum grades, or year standing. Checking these by hand means flipping between multiple pages and hoping you did not miss a condition buried three levels deep in an AND/OR expression. It gets even harder for students who are not following the traditional path through their program tree, whether that is because of co-op sequencing, repeated courses, transfer credits, late program changes, or simply taking courses in a different order than the default progression.

Existing tools do not really solve this. Carleton's public Course Scheduler lets you visualize timetables, but it knows nothing about your academic history. Third-party tools like Coursicle have the same limitation. The result is that students end up consulting three or four different sources, doing the cross-referencing in their heads or on a spreadsheet, and spending a lot of time brute-forcing combinations that fit their required courses, elective options, work hours,

and other constraints. Even after all that effort, there is still the question of whether they actually found every valid combination or whether they missed a schedule that would have fit better.

1.2 Project Objective

We set out to build a web application that connects a student's academic record to the course registration process. In concrete terms, the system needed to:

1. Parse a student's academic audit (HTML or PDF) to extract completed courses and figure out what they still need.
2. Check prerequisite eligibility for any course, using a structured representation of Carleton's prerequisite rules.
3. Generate all conflict-free timetable combinations for a set of courses, handling lectures, labs, and tutorials.
4. Show the results in an interactive timetable that students can browse and compare.

The application was scoped to run locally. We were not trying to integrate with Carleton Central's live registration system. The goal was to give students the Course Registration Numbers (CRNs) they need to enroll manually.

1.3 Report Overview

The rest of this report is organized as follows. Chapter 2 covers the standard engineering project considerations required by the department: health and safety, professionalism, project management, suitability for the Software Engineering degree, and who did what. Chapter 3 goes through the requirements and design, including what we originally proposed, what alternatives we considered, and why

we made the architectural choices we did. Chapter 4 gets into the implementation details for the prerequisite validator, scheduling algorithm, audit parser, database, API, and frontend. Chapter 5 covers testing and evaluation. Chapter 6 is a reflection on the project: what went well, what did not, and what we would change. The original project proposal is reproduced in Appendix C.

2 The Engineering Project

2.1 Health and Safety

This is a software project. There is no hardware, no lab equipment, and no physical safety risk. The main concern is data privacy: students upload their academic audits, which contain their name, student ID, and course history. Uploaded files are initially handled in memory, but PDF parsing requires a temporary file so `pdftotext -layout` can convert the document into structured text. Those temporary files are deleted immediately after parsing. If the user uses the import-and-save flow rather than preview-only parsing, the parsed student data is stored in the local PostgreSQL database, and the raw audit can also be stored for deduplication and later reference.

We did add a lightweight authentication layer for the current version of the system. The frontend starts with a login page, and the backend exposes JWT-based login and session-validation endpoints. That is enough for local development and for separating normal use from the admin tooling, but it is not production-grade security. All communication between the frontend and backend still happens over plain HTTP on localhost. A production deployment would need stronger credential handling, proper access control, and HTTPS, which was one of our original non-functional requirements (NFR6) that we still did not meet. Section 6 discusses this.

2.2 Engineering Professionalism

We tried to be honest throughout the project, both with our supervisor and with ourselves. When we fell behind on features, we told Prof. Talim in our meetings rather than pretending things were on track. When preference-based ranking turned out to be more work than we could do well in the remaining time, we chose to cut it rather than ship something half-baked. That felt like the right call, even though it meant the final product does not match the full scope of the proposal.

On the code side, we followed consistent TypeScript conventions, used descriptive names, and added comments where the logic was not obvious. The Prisma schema is the single source of truth for the data model, and the API follows RESTful conventions.

2.3 Project Management

The project ran across two terms (Fall 2025 and Winter 2026). We met weekly to go over what each person had done, talk through anything that was stuck, and figure out the next week's priorities. Between meetings, we communicated through a group chat. We used Git and GitHub for version control, with each person working on their own branch and merging into main.

During the Fall term, we focused on the foundation: backend infrastructure, database schema, the scheduling engine, prerequisite validation, and the initial frontend pages. One of the first real decisions we made was to split the project into a separate frontend (React + Vite) and backend (Express), rather than the unified Next.js app from the proposal. That was a project management decision

as much as a technical one. With four people touching the same codebase, we would have spent half our time resolving merge conflicts. The split let us work in parallel without stepping on each other.

In the Winter term, we shifted to integration, PDF audit parsing, elective recommendations, frontend polish, and this report. We tracked remaining work through shared lists and meeting notes instead of a formal issue tracker. For four people, that mostly worked. But there were times when it was not clear who owned a particular task, and a few things slipped through the cracks because of it. Chapter 6 says more about this.

More concretely, our tracking tools were lightweight rather than formal. We kept shared to-do lists for the current sprint of work, added checklists to meeting notes, and used GitHub branches and pull requests to show what had actually been completed. That combination let us answer three practical questions every week: what is done, what is blocked, and what still belongs to someone. It was not as structured as a Kanban board or issue tracker, but it was still a real progression-tracking process rather than ad hoc memory.

We did not set up CI/CD. Builds and tests were run manually. For a project of this duration and team size, that was workable, but it would not scale.

2.4 Justification of Suitability for Degree Program

All four team members are in the Software Engineering program, and this project used skills from across the curriculum. Here is how specific courses connected to the work we did:

Software Design and Architecture (SYSC 4120): The separated frontend/ backend architecture, the RESTful API contract, and the ORM-based data access layer are all applications of separation of concerns and layered design. Modeling prerequisites as a tree (using the Composite pattern) rather than flat text required the kind of data modeling we learned in this course.

Algorithms and Data Structures (SYSC 2100): The schedule generator is a constraint-satisfaction problem solved with backtracking search. The prerequisite validator uses recursive tree traversal. We analyzed both for time and space complexity and optimized the scheduler with pruning to get it under two seconds for typical inputs.

Database Management (COMP 3005): We designed the PostgreSQL schema in Third Normal Form with interrelated models for courses, prerequisites, sections, meetings, students, saved audits, users, and elective categories. The design uses foreign key constraints, unique constraints, and indexes, and Prisma ORM gave us type-safe database access from TypeScript.

Web Application Development (SYSC 4806): The frontend uses React 18 with component-based architecture, state management, and async API calls. The backend uses Express.js with RESTful routing, error handling, input validation, and middleware for file uploads and CORS.

Software Engineering Process: In practice, our process was closest to incremental development with regular feedback. We defined the main requirements early, divided the system into backend, frontend, database, and algorithm components, and then implemented and integrated those pieces over

multiple iterations. As the project progressed, we revised priorities, descoped features, and adapted the design, which is consistent with the engineering process methods taught in the class.

2.5 Individual Contributions

2.5.1 Project Contributions

Anas Ayoubi focused on the scheduling algorithm. He implemented the backtracking schedule generator, including support for multi-section courses and time-conflict checking.

Hakeem Khan handled data acquisition. He worked on scraping the source data and converting it into JSON so it could be loaded and used by the rest of the system.

Pietro Adamvoski handled the database schema and prerequisite logic. He worked on the Prisma models, including the database structure used for prerequisites, and contributed to the logic for checking whether a student meets course requirements.

Safiullah Rattar handled PDF parsing and search optimization. He worked on extracting course information from audit PDFs and on improving the search process so schedule generation could run more efficiently.

2.5.2 Report Contributions

- **Chapter 1 (Introduction)**: Pietro Adamvoski
- **Chapter 2 (The Engineering Project)**: All members (collaborative)
- **Chapter 3 (Requirements and Design)**: Pietro Adamvoski, Anas Ayoubi

- **Chapter 4 (Implementation)**: Pietro Adamvoski (database, prerequisites), Anas Ayoubi (scheduling algorithm), Safiullah Rattar (PDF parsing, search optimization), Hakeem Khan (data acquisition)
- **Chapter 5 (Testing and Evaluation)**: All members
- **Chapter 6 (Reflections)**: All members (collaborative)

3 Requirements and Design

This chapter covers what we set out to build and how we decided to build it. The focus is on the design process: what options we considered, why we picked what we picked, and where things changed from the original proposal.

3.1 Original Requirements

The proposal (October 2025) listed nine functional requirements (FR1–FR9) and six non-functional requirements (NFR1–NFR6). Tables 3.1 and 3.2 summarize the requirement categories discussed in the final system and how they ended up.

ID	Requirement	Status	Notes
FR1	Academic Audit Parsing (HTML/PDF)	Implemented	HTML fully supported; PDF supported via `pdftotext` CLI tool
FR2	Prerequisite Validation	Implemented	Tree-based recursive evaluation with grade checking
FR3	Manual Course Entry	Implemented	Search and select interface on Course Selection page
FR4	Elective Recommendation and Selection	Implemented	Audit parsing extracts elective categories, users can add elective tokens, the system resolves eligible options, and the schedule view supports elective swapping/regeneration
FR5	Schedule Generation	Implemented	Backtracking CSP solver with multi-section support
FR6	Preference-based Ranking	Not Implemented	Descoped; all valid schedules returned unranked
FR7	Interactive Timetable View	Implemented	Visual weekly grid with color-coded courses
FR8	Error and Conflict Feedback	Implemented	Prerequisite warnings, missing-section alerts, conflict detection

Table 1: Functional Requirements and Final Status

ID	Requirement	Status	Notes
NFR1	Performance (<10s generation, <5s load)	Met	Generation completes in <2s for 5 courses
NFR2	Usability (intuitive first-time use)	Partially Met	Three-step wizard flow; no formal SUS evaluation conducted
NFR3	Reliability (no crashes, graceful error handling)	Met	Try-catch on all routes; fallback to manual entry if parse fails
NFR4	Compatibility (Chrome, Firefox, Safari)	Partially Met	Tested primarily on Chrome; no formal cross-browser testing
NFR5	Maintainability (modular, documented)	Met	Separated modules, Prisma schema, architecture documentation
NFR6	Security (HTTPS)	Not Met	Application runs on localhost over HTTP; no deployment to production

Table 2: Non-Functional Requirements and Final Status

Most of the functional requirements were fully or partially implemented. The main feature we deliberately cut during the Winter term was FR6, preference-based ranking. Section 3.4 explains why.

3.2 System Architecture

3.2.1 Architectural Decision: Separated Frontend and Backend

The proposal called for a single Next.js 14 application. In that setup, the user interface and the server-side logic would both live in the same project instead of being split into separate frontend and backend codebases. We tried that in the first couple of weeks and quickly realized it was going to be painful for a four-person team. Everyone would be editing the same project, bumping into merge conflicts on shared files, and generally getting in each other's way.

So we split it. The frontend became a standalone React 18 app built with Vite and TypeScript. The backend became an Express.js server with its own TypeScript codebase, talking to PostgreSQL through Prisma. The two communicate over a REST API on localhost:3001.

This worked well. The frontend developer could build pages against mock data or a running backend without needing to touch any server code. The backend developers could test endpoints with curl or the CLI tool without opening a browser. Merge conflicts dropped significantly.

Vite's hot-module replacement was also faster in practice. Hot-module replacement is the feature that updates the browser immediately after a code change without rebuilding and reloading the whole page. With Vite, small frontend edits usually showed up almost right away. With Next.js, the feedback

loop felt slower because the development server had more framework overhead and often needed to do more work before showing the change. That speed difference mattered because frontend development involved a lot of small adjustments to layout, state handling, and component behavior. When each edit appears quickly, you can iterate much faster.

The tradeoff was losing Next.js features like server-side rendering, file-based routing, and integrated API routes. But we did not need SSR (this is a client-side tool, not a content site), and the explicit separation was worth more to us than the convenience of a unified framework.

3.2.2 High-Level Architecture

The system has three main runtime layers:

1. **Client layer (React + Vite)**: A single-page app that renders the UI, takes user input (audit uploads, course selections), and displays generated schedules. It talks to the backend via HTTP requests to `localhost:3001/api`.
2. **Server layer (Express + Node.js)**: The application server with the business logic. Its routes are organized into courses, students, schedules, audit, authentication, and admin management. The admin endpoints support section-data uploads, course and prerequisite editing, and elective-category management. All data access goes through Prisma Client.
3. **Data layer (PostgreSQL + Prisma)**: A relational database with all the course catalog data, section schedules, prerequisite rules, and student records. Prisma handles type-safe queries and schema migrations.

There is also a **CLI layer** for development and testing. Command-line scripts let you test the prerequisite validator, audit parser, and schedule generator without running the web app. This turned out to be one of the more useful things we built, because it let us iterate on algorithm logic much faster than going through the browser every time.

3.2.3 Design Patterns

A few standard patterns show up in the codebase:

MVC: The backend roughly follows Model-View-Controller. Prisma models are the data layer, Express route handlers are the controllers, and JSON responses are the view (there is no server-side HTML).

Repository pattern: All database access goes through Prisma Client. Route handlers never contain raw SQL. This keeps the data access layer testable and would make it easier to swap databases later.

Composite pattern: The prerequisite tree uses this. A PrerequisiteNode can be a leaf (COURSE, YEAR_STANDING, PERMISSION) or a composite (AND, OR) with child nodes. This lets us represent arbitrarily complex prerequisite expressions in a uniform way.

Strategy pattern: The audit parser picks a parsing strategy based on file type. The upload endpoint detects whether the file is HTML or PDF and dispatches to the right parser function. Both return the same `StudentAuditData` interface.

3.3 Database Design

3.3.1 Schema Design Process

The database schema went through several iterations. The first version stored prerequisites as plain text strings (e.g., “SYSC 3110 and SYSC 3020 or COMP 3005”). Easy to implement, impossible to evaluate programmatically. The system would have needed to parse that natural-language string every time it checked a prerequisite, and the phrasings in the Carleton calendar are inconsistent enough that reliable parsing was not realistic.

To make the evolution explicit, Table 3.3 summarizes the main schema iterations we went through before arriving at the final model.

Iteration	Idea	What Worked	Why It Failed / Changed
V1	Single `Course.prerequisiteText` field	Easy to scrape and display raw prerequisite text	Could not evaluate logic programmatically; natural-language prerequisite strings were too inconsistent to parse reliably at runtime
V2	Simple prerequisite junction table	Worked for basic "A and B" relationships	Could not represent nested logic such as `(A and B) or (C and D)`, minimum grades, year standing, or permission requirements
V3	Partially normalized prerequisite rows with type flags	Started separating course requirements from non-course requirements	Still awkward for nested expressions; evaluation logic became messy because grouping information was incomplete
V4	Self-referential `PrerequisiteNode` tree plus display text	Supports nested AND/OR groups, grade thresholds, year standing, permission, and recursive evaluation	This is the final design because it was the first version that correctly modeled the real prerequisite structures we found

Table 3: Database Schema Iterations for Prerequisite Modeling

We looked at three options:

1. **Flat text**: Store the raw prerequisite string. Simple, but requires runtime parsing of unstructured text. We rejected this because the parsing would be too fragile.
2. **Adjacency list**: Store prerequisites as rows in a junction table (course_id, required_course_id). This handles simple AND/OR cases but cannot

represent nested expressions like "(A AND B) OR (C AND D)." Many Carleton courses have prerequisites like this, so we ruled it out.

3. **Tree structure (what we went with)**: Store prerequisites as a tree of typed nodes in a self-referential table. Each node has a type (AND, OR, COURSE, YEAR_STANDING, PERMISSION), optional references to a required course or year level, and a parent pointer. This handles arbitrary nesting and covers every prerequisite pattern we found in the calendar.

The tree structure was the only option that correctly represents the full range of Carleton's prerequisite rules while still being straightforward to evaluate with a recursive traversal.

What made the prerequisite tree complex was not just the number of courses, but the shape of the logic. Carleton prerequisite statements often mix several kinds of conditions in the same expression: specific courses, minimum grades, year standing, and department permission. Some expressions also contain nested groups where one branch is a conjunction and another is a disjunction. A flat representation loses that structure. The traversal is therefore complex for the same reason: the algorithm is not just checking membership in a set, it is walking a logical expression tree and combining child results differently depending on whether a node is `AND`, `OR`, or a leaf condition.

3.3.2 Final Schema

The final schema grew beyond the original six-model design. In the current version, the main models are:

Model	Primary Key	Description
`Course`	`id` (e.g., "SYSC4907")	Course metadata: title, credits, description
`Prerequisite`	`courseId` (FK)	Raw prerequisite expression string for display
`PrerequisiteNode`	`id` (generated unique string ID)	Tree node: type, required course/year, parent pointer
`Section`	`id` (generated unique string ID)	Section metadata: CRN, section code, type, term, enrollment data
`SectionMeeting`	`id` (generated unique string ID)	Meeting rows for section day/time/date details
`Student`	`id` (generated unique string ID)	Student profile: student ID, name, program
`SavedAudit`	`id` (generated unique string ID)	Stored raw audit content and hash for deduplication
`User`	`id` (generated unique string ID)	Application login identity and role
`CourseCompletion`	`id` (generated unique string ID)	Junction table: student-course with grade
`ElectiveCategory`	`id`	Named elective bucket such as a program-specific elective category
`ElectiveCourse`	`id` (generated unique string ID)	Join table linking elective categories to valid course options

Table 4: Database Schema Summary

The schema is in Third Normal Form. We used the course code (for example, "SYSC4907") as the primary key for `Course` because that is already the identifier used throughout the Carleton calendar and the rest of the application. The other models use generated IDs where appropriate because they represent records such as sections, meetings, students, users, saved audits, prerequisite nodes, and course

completions, where there is no single natural key that is as clear or stable. The `PrerequisiteNode` table uses a self-referential foreign key (`parentId` referencing `id`) so prerequisite expressions can be stored as trees, and the elective subsystem uses `ElectiveCategory` plus `ElectiveCourse` to map degree-audit elective requirements to concrete course options.

3.3.3 Data Population

The database was populated from two sources:

1. **Timetable data**: Course section schedules (CRNs, times, instructors, locations) were provided by the project supervisor in a structured format. We wrote an import script to ingest this data. The database has 1,836 courses with section data and 7,680 individual sections.
2. **Prerequisite data**: We scraped prerequisite rules from the Carleton undergraduate calendar with a Node.js script. The scraper walked the calendar site, pulled out the prerequisite text for each course, and parsed it into tree form. The database has 322 courses with parsed prerequisite trees, 348 courses total.

Appendix B gives a compact summary of the final database statistics.

3.4 Descoped Features and Rationale

FR6: Preference-based ranking. The proposal described a scoring system that would rank schedules based on preferences like avoiding morning classes or keeping a day free. We cut this for two reasons. First, we underestimated how much work the core scheduling engine (FR5) and prerequisite validation (FR2) would need, particularly handling multi-section courses and evaluating the

prerequisite trees. Those took up most of the Winter term. Second, the system already returns up to 50 valid schedules that you can flip through visually, which gets you most of the way there even without automated ranking. Building a proper preference UI, a scoring algorithm, and testing it all was more than we could do well in the time we had left.

The misjudgment was mostly in the algorithmic core. On paper, schedule generation looked like a straightforward combination problem and prerequisite checking looked like a standard tree walk. In practice, both expanded. Schedule generation needed support for lecture/lab/tutorial combinations, internal conflicts within one course, pruning, and a cap on result count. Prerequisite validation needed minimum grades, year standing, permission nodes, and parser support for nested expressions from the calendar. Those parts were more foundational than ranking, so once they consumed more time than expected, ranking became the feature that had to be dropped.

4 Implementation

This chapter goes through the implementation of each major component. The emphasis is on the algorithms, data structures, and places where we had to make judgment calls.

4.1 Prerequisite Validation Engine

4.1.1 Problem Statement

Given a student's transcript (completed courses with grades) and a target course, determine whether the student satisfies all prerequisites. Prerequisite rules at Carleton come in several forms:

- Simple requirements: "SYSC 3110"
- Conjunctions: "SYSC 3110 and SYSC 3020"
- Disjunctions: "SYSC 3110 or COMP 3005"
- Nested expressions: "(SYSC 3110 and SYSC 3020) or (COMP 3005 and COMP 3007)"
- Minimum grades: "SYSC 2004 with a minimum grade of C+"
- Year standing: "Fourth-year standing"
- Special permission: "Permission of the Department"

4.1.2 Data Structure

Prerequisites are stored as a tree of typed nodes in the `PrerequisiteNode` table. Each node is one of five types:

- **AND**: All children must be satisfied (logical conjunction).
- **OR**: At least one child must be satisfied (logical disjunction).

- **COURSE**: A specific course must be completed, optionally with a minimum grade.
- **YEAR_STANDING**: The student must have reached a specified year level.
- **PERMISSION**: The student must have special permission (treated as a boolean flag).

For example, the prerequisite "SYSC 3110 and (COMP 3005 or COMP 3007)" is represented as:

```
         AND (root)
        /         \
   COURSE          OR
  SYSC3110        /    \
            COURSE    COURSE
           COMP3005  COMP3007
```

4.1.3 Algorithm

The validation algorithm is a recursive depth-first traversal of the tree:

```
evaluateNode(node, transcript):
  case AND:
    return ALL(evaluateNode(child) for child in node.children)
  case OR:
    return ANY(evaluateNode(child) for child in node.children)
  case COURSE:
    hasCourse = node.requiredCourseId in transcript.completedCourses
    if not hasCourse: return false
```

```
    if node.requiredGrade:
      return transcript.grades[node.requiredCourseId] >=
node.requiredGrade
    return true
  case YEAR_STANDING:
    return transcript.currentYear >= node.requiredYear
  case PERMISSION:
    return transcript.hasSpecialPermission
```

The time complexity is O(n) where n is the number of nodes in the prerequisite tree, since each node is visited exactly once. The space complexity is O(h) where h is the height of the tree, due to the recursion stack.

We wrote this evaluator from scratch. We did not use a rule engine or a third-party library for prerequisite logic because the shape of the problem was tightly coupled to our own schema and to the exact node types we needed to support.

The implementation went through several iterations: first a simple course-only checker, then support for `AND` and `OR` groups, then grade thresholds, then year-standing and permission leaves, and finally better error reporting so the frontend could explain why a course failed validation.

4.1.4 Why the Traversal Was Non-Trivial

The traversal looks short in pseudocode, but the real difficulty is that each node type changes the meaning of the subtree below it. A `COURSE` node asks whether one requirement is met. An `AND` node asks whether all of its children are true. An `OR` node asks whether at least one branch is true. Once minimum grades and year

standing are included, the algorithm is no longer just checking whether a student has taken a course; it must interpret different kinds of evidence from the transcript and combine them correctly. That is why the tree representation and the traversal algorithm had to be designed together.

4.1.5 Grade Comparison

Grades are mapped to numbers (A+ = 12, A = 11, down to F = 0) so we can compare them. If a grade cannot be parsed (transfer credits marked "TR" or "SAT," for instance), the system treats the requirement as satisfied. If the system has to guess, we would rather let a student through than incorrectly block them from a course they are probably allowed to take.

4.1.6 Design Decision: Recursive vs. Iterative

We used recursion because prerequisite rules are stored as trees, and recursion is an intuitive way to process that kind of structure. An iterative version was possible, but it would have needed extra code to keep track of which nodes had already been visited, which ones still needed to be checked, and how the child results should be combined at each AND or OR node. That makes the logic harder to follow. Since prerequisite trees are small (typically under 20 nodes), recursion does not add much risk here. The call depth stays shallow, so there is no realistic chance of running into stack overflow problems.

4.2 Schedule Generation Algorithm

4.2.1 Problem Definition

Given n courses, where each course has multiple sections (lectures, labs, tutorials) at specific times, find all timetable combinations where:

1. Every course has all its required section types covered (e.g., one LEC and one LAB).
2. No two selected sections overlap.

This is a constraint satisfaction problem (CSP).

4.2.2 Algorithm Selection

We considered three approaches:

1. **Brute force**: Generate every possible combination, then filter for valid ones. The search space grows as $O(s^n)$ where s is the average number of section combinations per course and n is the number of courses. Even with modest inputs this becomes infeasible, so we ruled it out.
2. **Genetic algorithm**: Start with a set of candidate schedules, score them, and repeatedly modify and combine them to try to produce better ones over time. We rejected this because our problem size is small enough (4–6 courses in practice) that we can search directly for exact answers. Genetic algorithms are better suited to very large search spaces where finding a good answer is enough. In our case, we wanted all valid schedules, not just one or two decent ones, and a genetic algorithm could easily miss some of them.
3. **Backtracking with constraint propagation (what we used)**: Explore the possible schedules in an organized way, one course at a time. Whenever a partial schedule already has a time conflict, stop exploring that path immediately and move on to the next one. This lets the algorithm find all valid schedules without wasting time on combinations that are already invalid.

We also wrote this algorithm from scratch. There are libraries and research tools for timetabling, but they are usually aimed at institutional scheduling problems with different constraints than ours. Our problem was narrower: given a small set of student-selected courses, enumerate all valid schedules and stop after a reasonable cap. Building our own solver let us control the search order, pruning rules, section grouping, and output format.

4.2.3 Implementation

The algorithm works in three phases:

Phase 1: Data preparation. For each selected course, fetch all available sections from the database and group them by type (LEC, LAB, TUT). Compute the Cartesian product across section types to get every possible combination for that course. A course with 3 lectures and 2 labs produces 6 LEC+LAB combinations. Before the search even starts, we filter out any combination where a course's own sections conflict with each other (e.g., a lecture and a lab that overlap).

Phase 2: Recursive backtracking. Process courses one at a time. For the current course, try each valid section combination. Check whether any section in the combination conflicts with sections already assigned to previous courses. If there is no conflict, add the combination to the partial schedule and recurse to the next course. If all courses have been assigned, record the schedule as a solution. Then backtrack: remove the last assignment and try the next combination. Stop after finding 50 valid schedules.

Phase 3: Prerequisite check. After generating schedules, optionally validate that the student meets the prerequisites for each course using the validator from Section 4.1. This information goes into the API response so the frontend can show prerequisite warnings.

4.2.4 Conflict Detection

Two sections conflict if they are on the same day and their time intervals overlap. With start and end times in minutes since midnight, the overlap condition is:

$$\text{start}_1 < \text{end}_2 \wedge \text{start}_2 < \text{end}_1$$

This is O(1) per pair of time slots. Day strings like "MWF" are expanded into individual days before comparison.

4.2.5 Backtracking Example

Suppose the student selects three courses:

- `SYSC4001` with two valid section combinations: `A1` and `A2`
- `COMP3005` with two valid section combinations: `B1` and `B2`
- `ELEC4700` with one valid section combination: `C1`

Assume `A1` conflicts with `B1`, `A2` conflicts with `C1`, and the other pairings do not conflict. The search proceeds like this:

```
start: []
try A1 -> [A1]
  try B1 -> conflict, reject branch
  try B2 -> [A1, B2]
    try C1 -> valid, record [A1, B2, C1]
  backtrack from B2
backtrack from A1
```

```
try A2 -> [A2]
  try B1 -> [A2, B1]
    try C1 -> conflict, reject branch
  backtrack from B1
  try B2 -> [A2, B2]
    try C1 -> conflict, reject branch
  backtrack from B2
backtrack from A2
```

The important point is that the solver never keeps exploring after a partial schedule is already invalid. As soon as `A1 + B1` conflicts, the entire subtree under that partial assignment is discarded. That pruning is what makes backtracking practical.

4.2.6 Deterministic Behavior

The generator is deterministic. For a fixed input set of courses, sections, and completed-course data, it visits the search space in the same order and therefore returns the same schedules in the same order every time. That means if 50 students upload the exact same audit and make the exact same selections, the system will generate the same result set for all 50 of them. This is useful for debugging and for consistent user expectations.

4.2.7 Performance

Courses	Avg. Sections/ Course	Estimated Search Space	Observed Time
1	8	8	$<$ 100ms
2	6	48	$<$ 200ms
3	5	240	$<$ 500ms
4	4	960	$<$ 1s
5	3	2,430	$<$ 2s

Table 5: Schedule Generation Performance

Pruning eliminates roughly 99.99% of the search space for typical inputs. Most partial schedules get abandoned early because of conflicts, so the algorithm rarely comes close to exploring the full space. The worst-case complexity is $O(b^d)$ where b is the average number of section combinations per course and d is the number of courses, but in practice the pruning makes this much faster.

We cap requests at 8 courses to avoid unusually large inputs that would create a very large search space and make schedule generation take too long.

4.3 Academic Audit Parsing

4.3.1 HTML Parsing

The main audit parsing method processes the HTML file that students can save from the Carleton Central academic audit page. The parser uses regular expressions to extract:

- **Student ID**: Matched by `(\d{9})`.
- **Student name**: Matched from header text in "LastName, FirstName" format.

- **Program**: Extracted from "Bachelor of Engineering - …" text.
- **Completed courses**: Pulled from HTML table rows with class `takenCourse`, capturing the course code, term, and grade.
- **Current courses**: Same structure, identified by grade value "CUR".
- **Degree requirements**: Extracted from `subrequirement` div elements, with credit counts, completion status, and the courses that satisfy each requirement.

We used regular expressions instead of a DOM parsing library (Cheerio was in the proposal) after looking at the actual HTML. Carleton's audit pages are generated by a consistent server-side system and follow predictable patterns. A DOM parser would be more robust against structural changes, but it would add a dependency for something that regex handles fine given how regular the input is.

4.3.2 PDF Parsing

PDF parsing was a secondary objective in the proposal because PDFs are inherently harder to extract structured data from. We first tried pdf-parse (a JavaScript library), but the text extraction was unreliable for the multi-column layout in Carleton audit PDFs. Text would come out jumbled or with columns interleaved.

What worked was `pdftotext` (from the Poppler library) with the `-layout` flag, which preserves the spatial arrangement of text on the page. The backend writes the uploaded PDF to a temp file, runs `pdftotext` as a child process, reads the output, and parses it with regex. Temp files are cleaned up immediately after, even if an error occurs.

The PDF parser matches course lines like `"21FA  CHEM 1101  0.50  A+"` using:

```
/^\s*(\d{2}(?:FA|WN|SU))\s+([A-Z]{3,4})\s+(\d{4})\s+([\d.]+)\s+([A-Z+\-]+)/gm
```

This pattern looks for course lines that follow the usual audit format. It pulls out the term, subject code, course number, credit value, and grade. That gives the parser the exact pieces it needs from each course entry without having to process the whole line manually.

It handles repeated courses (student retook a course) by keeping the most recent grade, and drops courses with F, UNS, or NR grades.

The hardest part of audit extraction was that not all information appears in a single clean, machine-readable format. In HTML audits, most data lives in predictable tags and classes, which made regex extraction feasible. In PDFs, the same information is flattened into positioned text, so columns can interleave, spacing can vary, and section headers may break the sequence of course rows. It was especially hard when:

- the audit used a multi-column layout and `pdf-parse` mixed text from both columns together,
- repeated-course entries appeared in separate parts of the audit,
- requirement sections included narrative text that looked similar to course rows,
- grades such as `CUR`, `TR`, or `SAT` needed different handling from standard letter grades,

- elective requirement blocks described categories rather than concrete course codes.

Those cases are why PDF parsing remained less robust than HTML parsing and why we treated PDF support as secondary until `pdftotext -layout` gave us a stable enough text representation.

4.3.3 Design Decision: `pdftotext` vs. JavaScript Library

Using a system-level CLI tool instead of a JavaScript library was not what we planned. But Poppler's text extraction is significantly better than the JS alternatives we tried, and it handles multi-column layouts correctly. The downside is a system dependency: `pdftotext` has to be installed on the host machine, which hurts portability. For a local development tool, that was an acceptable tradeoff. For production, we would either bundle it in a Docker container or revisit the JS libraries.

4.4 Backend API

The Express backend has four groups of endpoints:

Method	Endpoint	Description
GET	`/api/courses/search?q=&limit=`	Search courses by code or title
GET	`/api/courses/:courseId`	Get course details with prerequisites
GET	`/api/courses/:courseId/sections`	Get all sections for a course
GET	`/api/students/:studentId`	Get student profile and completions
POST	`/api/students/:studentId/validate`	Validate prerequisites for a course
GET	`/api/students/:studentId/eligible`	Get courses student is eligible for
POST	`/api/schedules/generate`	Generate conflict-free schedules
POST	`/api/audit/upload`	Upload and save audit data
POST	`/api/audit/parse`	Parse audit without saving (preview)
POST	`/api/audit/suggest-courses`	Suggest courses from remaining requirements
POST	`/api/auth/login`	Create authenticated local session
GET	`/api/admin/electives`	List elective categories and course options
PUT	`/api/admin/courses/:courseId/prerequisites`	Edit prerequisite data in admin UI

Table 6: REST API Endpoints

The server runs on port 3001 with CORS configured for the frontend dev server. File uploads use the Multer middleware with a 10 MB limit and file type validation (HTML and PDF only). All route handlers catch errors and return appropriate HTTP codes (400 for bad input, 404 for missing resources, 500 for server errors) instead of letting unhandled exceptions crash the process. Appendix A includes example API requests and responses for the schedule generation endpoint.

4.5 Frontend Interface

The current frontend starts with authentication and then exposes a three-step planning wizard, plus a separate admin page for data management.

The user flow is linear by design because schedule generation only makes sense after the system has enough input data. Before a student can use the generator, the system needs either: (1) an academic audit file (`.html` or `.pdf`) so it can infer completed courses and remaining requirements, or (2) a manual course selection if the student skips the audit step.

```
Student opens app
     |
     v
Sign in
     |
     v
Upload audit (.html or .pdf) OR skip import
     |
     v
Parse academic history
     |
     v
Confirm extracted student data
     |
     v
Search/select courses and electives
     |
     v
Generate schedules
     |
     v
Browse timetable + copy CRNs
```

Listing 1: High-Level User Flow Before and During Schedule Generation

4.5.1 Login Page

The current version of the application opens on a login page backed by the `/api/auth/login` endpoint. This is a lightweight JWT-based flow intended for the local application context. It is enough to gate access to the scheduling and admin interfaces, but it is not meant to be presented as a production-ready authentication system.

4.5.2 Page 1: Academic History Import

The first page lets users upload an academic audit (HTML or PDF) or skip ahead to manual course selection. When a file is uploaded, it goes to the `/api/audit/parse` endpoint, and the page shows the extracted info: student name, ID, program, completed courses, current courses, and remaining requirements. The user confirms the parsed data before moving on. If parsing fails or the user does not have an audit file, they can proceed without it, and prerequisite indicators are hidden.

The technologies active in this step are the React upload form on the frontend, Multer on the Express backend for multipart file handling, the parser dispatch logic that chooses HTML or PDF processing based on file type, and PostgreSQL/Prisma only if the user decides to save the parsed audit rather than just preview it.

4.5.3 Page 2: Course Selection

This page has a search bar (backed by `/api/courses/search`) with 300ms debounce to avoid hammering the API on every keystroke. Users can also add courses from their remaining requirements list, which comes from the audit

parsing step. Each selected course shows its prerequisite status: green if met, amber if not, hidden if there is no audit data. Elective categories (like "Computer Science Elective" or "Complementary Studies") show up as selectable tokens. Selecting one creates an elective slot token, and the system resolves that token into concrete course choices using the elective category catalog, term availability, and prerequisite status.

The technologies active here are React state management for the selected courses, shadcn/ui components for the search and token interface, the `/api/courses/search` and `/api/students/:studentId/validate` endpoints on Express, and Prisma queries against the course and prerequisite tables.

4.5.4 Page 3: Schedule Review

Generated schedules appear in a weekly timetable grid. Courses are color-coded, and each block shows the course code, section code, time, and location. You flip between schedules with prev/next buttons ("Schedule 1 of 50"). A sidebar lists all sections in the current schedule with their CRNs, and each CRN can be copied to the clipboard for entry into Carleton Central.

The technologies active in this final step are the backtracking generator on the backend, the prerequisite validator for warning badges, the REST response serialization, and the React timetable UI that maps returned section times into a visual weekly grid.

If elective categories were selected, the sidebar also has an elective panel where you can browse available courses for each category, see which ones have sections and met prerequisites, and swap electives. Swapping regenerates the schedules, so

electives are part of the actual schedule-generation workflow rather than a placeholder suggestion feature.

4.5.5 Admin Page

The current frontend also includes an admin interface. It provides separate tabs for section-data uploads, course and prerequisite editing, and elective-category management. This page exists mainly to support maintaining the local dataset and correcting prerequisite or elective mappings without editing the database manually.

4.5.6 UI Technology

The frontend uses 48 shadcn/ui components (built on Radix UI) for the UI elements, TailwindCSS v4 for styling, and Lucide icons. Application state (current step, parsed audit data, selected courses, generated schedules, elective selections) lives in the top-level `App.tsx` component and gets passed down as props, following React's one-way data flow.

5 Testing and Evaluation

5.1 Testing Approach

We did not use an automated test framework. No Jest, no Vitest. This is probably the biggest shortcoming of the project, and Chapter 6 discusses it. All testing was done manually during development.

Because academic audits contain personally identifiable information and real course histories, our audit-parsing tests were done in the minimum responsible way. We only tested with audits voluntarily provided by team members, and we did not build or keep a large stored corpus of real student audits. That limited our test coverage, but it was the more responsible choice for a local student project without a formal data-governance process.

5.1.1 Manual Functional Testing

We tested each feature by hand against known inputs:

Prerequisite Validation: We ran the validator against a range of scenarios:

- Courses with a single prerequisite (e.g., SYSC 3110 requires SYSC 2004).
- AND prerequisites (e.g., needing both SYSC 3110 and SYSC 3020).
- OR prerequisites (e.g., COMP 3005 or COMP 3007).
- Nested expressions (e.g., "(A and B) or (C and D)").
- Minimum grade requirements (e.g., SYSC 2004 with minimum C+).
- Year-standing requirements (e.g., fourth-year standing).
- Courses with no prerequisites (should always pass).

The CLI tool was invaluable for this. We could test dozens of scenarios in a few minutes without touching the browser.

Schedule Generation: We tested with:

- Single-course inputs (trivial case: should return all sections).
- Two-course inputs with known section times (manually verified the output).
- Multi-section courses (LEC+LAB) to check that all valid combinations came through and internal conflicts were filtered.
- Stress tests with 6–8 courses for performance and the 50-schedule cap.
- Edge cases: courses with no available sections, sections at identical times.

We also tested behavior across different student profiles. One useful case was a student with relatively few mandatory courses but many elective choices. That kind of input can produce a much larger number of valid timetables than a tightly constrained engineering term, so it was a good way to expose performance issues and verify that the 50-schedule cap worked as intended. Another useful case was a very constrained student profile where many selected courses shared overlapping lecture times, because that quickly exposed false positives or false negatives in conflict detection.

During this testing we hit several bugs that shaped the final version of the code:

- course combinations that were valid individually but contained an internal lecture/lab overlap,
- duplicate-looking schedules caused by different section-group ordering,
- prerequisite warnings not matching the final selected elective after a swap,
- repeated-course audit entries keeping the wrong grade instead of the most recent one,

- PDF parsing failures when text extraction merged both columns together.

Audit Parsing: We tested the HTML parser with real academic audits from team members. The PDF parser was tested with the same audits saved as PDFs. We manually compared the outputs to verify that courses, grades, and student info were extracted correctly.

This was intentionally limited testing because of privacy and confidentiality. If broader access to anonymized audits had been possible, the next step would have been to build a curated test corpus covering multiple degree programs, students with transfer credits, repeated courses, incomplete grades, large elective blocks, and formatting variation across different audit exports. That would have let us turn the parser into a more systematic regression-tested component.

Frontend Integration: We tested the full flow end-to-end in the browser: upload audit, select courses, generate schedules, browse timetables, copy CRNs.

5.1.2 CLI Testing Tool

The project includes a command-line interface (`cli/` directory) that lets you test backend logic without the web app. It has commands for:

- Validating prerequisites for any course against a given transcript.
- Parsing an audit file and printing the extracted data.
- Running the schedule generator for a set of courses and printing results.

This was one of the better decisions we made. Having immediate feedback on algorithm behavior without the overhead of the full web stack saved a lot of time. It also mattered for iteration speed. Because both the prerequisite evaluator and the schedule generator were written from scratch, we needed a fast way to run

the same scenarios repeatedly while refining the algorithms. The CLI made that possible.

5.2 Requirements Verification

Requirement	**Verification Method**	**Result**
FR1: Audit Parsing	Tested with team members' actual audits (HTML and PDF)	Pass — correct extraction of courses, grades, and student info
FR2: Prerequisite Validation	CLI testing with known prerequisite trees	Pass — correctly evaluates AND, OR, COURSE, YEAR_STANDING nodes
FR3: Manual Course Entry	UI testing: search, select, remove courses	Pass — courses can be searched, added, and removed
FR4: Elective Recommendation	Tested audit-derived elective categories, token selection, automatic resolution, and swap/regeneration workflow	Pass — elective categories are integrated into course selection and schedule review
FR5: Schedule Generation	Tested with 1–8 course combinations	Pass — all valid schedules generated, no false conflicts
FR6: Preference Ranking	N/A	Not implemented
FR7: Timetable View	Visual inspection of generated schedules in browser	Pass — correct time placement and color coding
FR8: Error Feedback	Tested with invalid inputs, missing sections, unmet prereqs	Pass — appropriate error messages and warnings displayed

Table 7: Functional Requirements Verification Matrix

5.3 Limitations of Testing Approach

The lack of automated tests is the biggest testing gap. Without a test suite, there is no regression protection. A change to one component can break another and nobody notices until someone happens to test that path manually. We relied on code review and manual testing to catch errors, which works but does not scale. If the project were to continue, we would add:

1. Unit tests for the prerequisite evaluator (it is a pure function with clear inputs and outputs, easy to test).
2. Unit tests for conflict detection.
3. Integration tests for the API endpoints using a test database.
4. End-to-end tests with something like Playwright or Cypress.

We also never did the User Acceptance Testing with student volunteers that the proposal described. The system was tested only by the four of us. We know it too well to evaluate usability objectively. Even informal testing with a few peers outside the team would have been useful.

6 Reflections

This chapter is a straightforward look at the project: what worked, what did not, and what we would change if we had to do it again.

6.1 Achievement of Original Objectives

The proposal said we would build an application that automates requirement analysis, validates prerequisites, generates conflict-free timetables, and ranks them by preference. In the end, we got three of those four done:

1. **Automated requirement analysis**: The system parses academic audits in HTML and PDF format, extracts completed courses and degree requirements, and figures out what courses a student still needs. Done.
2. **Prerequisite validation**: The tree-based validator handles AND/OR logic, grade thresholds, year-standing requirements, and the various other patterns in the Carleton calendar. Done.
3. **Conflict-free timetable generation**: The backtracking solver finds all valid schedules for a course selection, handles multi-section courses, and finishes in under 2 seconds for typical inputs (well within the 10-second target from NFR1). Done.
4. **Preference-based ranking**: Not implemented. Users browse unranked schedules manually.

The elective feature (FR4) ended up more complete than we first expected. The parser extracts elective requirements from the audit, the course-selection page lets users add elective-category slots, the scheduler resolves those slots into concrete courses, and the schedule-review page supports swapping electives and

regenerating schedules. There is still room to improve the breadth of elective catalogs and the polish of the admin tooling, but the core elective workflow is implemented.

6.2 What Changed and Why

Quite a bit changed between the proposal and the final system:

Technology stack: The biggest change was going from Next.js to React+Vite+Express. Section 3.2.1 covers the details. The short version is that we needed the split so four people could work without constantly stepping on each other, and that part worked.

Audit parsing approach: The proposal said Cheerio for HTML and a JavaScript library for PDF. We ended up using regex for HTML and `pdftotext` for PDF. Neither was the plan, but both were driven by what actually worked when we tried parsing real audit files. The JS PDF libraries produced garbled output. The HTML was regular enough that regex worked fine, so that is what we used.

UI library: The proposal mentioned Material-UI or Ant Design. The final version used Radix UI/shadcn instead. That was mostly a preference call and it did not change the core functionality.

Deployment: The proposal said we would deploy to Vercel. We never did. Partly because the frontend/backend split made Vercel less convenient (it is optimized for Next.js), and partly because deployment was not a priority given everything else that still needed doing. This means NFR6 (HTTPS) was not met.

Descoped features: FR6 (ranking) got cut. We underestimated how much time the core features would take, especially multi-section schedule generation and the prerequisite tree system.

6.3 Team Dynamics

For the most part, the team worked well together. Splitting responsibilities by strength (Hakeem on data acquisition, Pietro on database schema and prerequisite logic, Anas on the scheduling algorithm, Safi on PDF parsing and search optimization) kept us out of each other's way and let people focus on the parts they were strongest at. Weekly meetings were enough to keep everyone lined up.

That said, things got shakier during the Winter term, especially around integrating the frontend with the backend API. Without a proper task tracker (we never set up GitHub Issues or anything like it), it was sometimes unclear who owned what. A few tasks fell through the cracks because everyone assumed someone else was handling them. In the last few weeks, when multiple features were being finished simultaneously, better tracking would have helped.

6.4 Hindsight Decisions

If we started over knowing what we know now:

Write tests from the beginning. The lack of automated tests made refactoring scary and led to bugs that only surfaced during manual testing late in the process. Even basic unit tests for the prerequisite evaluator and conflict detection would have saved us time in the long run. We kept saying we would add tests later, and later never came.

Use a task tracker. GitHub Issues with a project board would have been easy to set up and would have made ownership clearer. Our shared-list approach worked until it didn't.

Cut features earlier. By the end of the Fall term, we already had a pretty good sense that ranking was slipping. We still waited too long to say that out loud and cut it. Making that decision earlier would have given us more time to polish the parts we actually finished.

Do user testing. The proposal described UAT with student volunteers. We never did it. Even asking a few classmates to try the app and watching where they got confused would have been useful.

Set up Docker. We occasionally ran into environment issues: different Node.js versions, missing system dependencies like `pdftotext`. A Docker Compose file would have prevented all of that.

6.5 Overall Assessment

The Smart Scheduler works. You can upload an audit, it figures out what you have taken, you pick courses, and it generates valid schedules. The prerequisite tree and the backtracking solver are the parts we feel best about. Those are the pieces that solve the real planning problem.

But the project has gaps. No automated tests. Two features that never made it past the proposal. No formal usability evaluation. These are not from lack of effort. We underestimated how much work the core features would take and did not leave enough margin for everything else.

What we got out of this project was not just the prototype. We also learned a lot from the process around it: making design decisions with tradeoffs, changing plans when our assumptions were wrong, coordinating work across a team, and being honest when things did not go the way we expected. We wanted the report to capture that part too.

7 References

1. I. Sbeity, M. Dbouk, and H. Kobeissi, "Combining the analytical hierarchy process and the genetic algorithm to solve the timetable problem," *arXiv preprint*, Sep. 2014. Available: https://arxiv.org/abs/1409.2650
2. L. Zhang, *Solving the timetabling problem using constraint satisfaction programming*, MInfoSys thesis, University of Wollongong, Australia, 2005. Available: https://ro.uow.edu.au/articles/thesis/27827784
3. T. Schaub, "Answer set programming as a modeling language for course timetabling," *Theory and Practice of Logic Programming*, vol. 8, no. 5, pp. 541–564, Sep. 2008.
4. R. K. Thakur, N. K. Agrawal, and P. Kumar, "A practical approach to college timetable scheduling," *Mathematical Modeling and Computing*, vol. 11, no. 3, pp. 710–719, 2024.
5. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
6. R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2018.
7. M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2003.
8. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024. Available: https://www.postgresql.org/docs/16/
9. Express.js Contributors, *Express.js Documentation*, 2024. Available: https://expressjs.com/

10. Prisma Data, Inc., *Prisma Documentation*, 2024. Available: https://www.prisma.io/docs/

H Appendix A: API Reference

The complete REST API is documented in Section 4.4. The base URL is `http://localhost:3001/api`. All endpoints return JSON responses. Error responses follow the format:

```
{
  "error": "Error Type",
  "message": "Human-readable description"
}
```

H.0.1 Schedule Generation Request Example

```
POST /api/schedules/generate
{
  "courseIds": ["SYSC4907", "SYSC4001", "ELEC4705"],
  "maxSchedules": 50,
  "completedCourses": ["SYSC3110", "SYSC3020", "COMP3005"]
}
```

H.0.2 Schedule Generation Response Example

```
{
  "schedules": [
    {
      "id": 1,
      "selections": [
        {
          "courseId": "SYSC4907",
```

```
          "section": {
            "crn": "30123",
            "sectionCode": "A",
            "sectionType": "LEC",
            "days": "TR",
            "startTime": "1135",
            "endTime": "1325",
            "instructor": "Talim, J."
          }
        }
      ],
      "hasConflicts": false,
      "prerequisiteInfo": {
        "SYSC4907": {
          "isValid": true,
          "expressionEvaluated": "..."
        }
      }
    }
  ],
  "count": 42,
  "requested": 3,
  "maxSchedules": 50
}
```

I Appendix B: Database Statistics

Metric	Value
Total courses in database	348
Courses with prerequisite trees	322
Courses with section data	1,836
Total sections	7,680
Prerequisite node count	2,000
Maximum prerequisite tree depth	4 levels
Average sections per course	4.2

J Appendix C: Original Project Proposal

The following pages reproduce the original project proposal

(`G67_SmartScheduler.pdf`, October 17, 2025) as submitted.

2025

Smart Scheduler for Carleton Engineering Students

Project Proposal



Smart Scheduler

Report Citation

Smart Scheduler for Carleton Engineering Students.

Smart Scheduler.

October 17, 2025.

G67 - Proff. Jerome Talim

Team Members:

Anas Ayoubi (101226450)

Hakeem Khan (101233039)

Pietro Adamvoski (101238885)

Safiullah Rattar (101245709)

Smart Scheduler
October 17, 2025

ABSTRACT

The course scheduling process for upper-year engineering students at Carleton University is inefficient and prone to error. The existing system requires students to manually select courses without contextual validation against their academic progress and provides no integrated tools for discovering and scheduling suitable electives. This project proposes the design and implementation of an intelligent course scheduler to address these deficiencies. The system will introduce two primary innovations: (1) the ability to parse a student's academic audit (in PDF or HTML format) to automatically determine completed courses and identify outstanding degree requirements, and (2) an integrated elective recommendation engine that suggests valid, schedule-compatible options. Using this data, the scheduler will generate all conflict-free timetables and rank them according to user-specified preferences. The solution will be a full-stack application developed with Next.js and TypeScript, supported by a PostgreSQL database. Project success will be measured by the accuracy of prerequisite validation, the efficiency of schedule generation, and overall usability.

TABLE OF CONTENTS

Abstract 2
1.0 Introduction 4
2.0 Background 6
3.0 Project Objective 14
3.1 Table 1: Functional Requirements 7
3.2 Table 2: Non-Function Requirements 9
3.3 Success Metrics 10
4.0 Proposed System Architecture 10
4.1 High-Level Design 10
4.2 Technology Stack 10
4.3 User Workflow 12
5.0 Methodology 14
5.1 Audit Parsing and Prerequisite Validation 14
5.2 Scheduling and Ranking Engine 15
5.3 Database Design 16
5.4 Security and Privacy Considerations 17
5.5 Testing Strategy 17
6.0 Relation to Degree Program 17
7.0 Collective Skills of the Group 19
8.0 Project Timeline and Milestones 20
8.1 Phased Timeline 20
8.2 Work Breakdown Structure 21
9.0 Risks and Mitigation Strategies 24
9.1 Technical Risks 25
9.2 Data-Related Risks 25
9.3 Project Management Risks 26
10.0 Conclusion 28
Acknowledgment 29
References 30

1.0 INTRODUCTION

For upper-year engineering students at Carleton University, constructing a valid course timetable is a complex manual task fraught with inefficiency. The university's current registration system, Carleton Central, functions primarily as an enrollment platform, leaving two significant strategic planning gaps:

1. **Manual Prerequisite and Requirement Validation:** Students must manually cross-reference their academic audit with the university calendar to select courses. This process is disconnected, time-consuming, and susceptible to errors in prerequisite checking.
2. **Lack of Elective Guidance:** The system offers no mechanism to recommend eligible technical or complementary electives. Students must consult separate university webpages to find lists of electives and then manually test each one for schedule compatibility through trial and error.

This project proposes the development of the Carleton Smart Scheduler, a web-based application designed to solve these problems by automating and optimizing the course planning process. The primary goal is to provide students with a centralized, intelligent tool that bridges the gap between their academic progress and the course registration system.
The key objectives of this project are:

1. **Automate Requirement Analysis:** To develop a robust parsing engine capable of interpreting student academic audits (in HTML or PDF format) to accurately identify completed courses and outstanding degree requirements. This provides the academic context currently missing from the manual selection process.
2. **Implement an Elective Recommendation Engine:** To build a feature that automatically identifies and suggests eligible electives that are compatible with the student's core course schedule and program requirements.
3. **Generate Conflict-Free Timetables:** To create a scheduling engine that generates all possible valid timetables based on a student's selected courses, accounting for lectures, labs, and tutorials.

4. **Implement Preference-Based Ranking:** To allow users to define scheduling preferences (time of day, schedule density) and use a scoring model to rank the generated timetables accordingly.

The scope of this project encompasses the design, implementation, and testing of a full-stack web application, including a front-end interface, a backend for business logic, and a database for course data. The system will not integrate directly with Carleton's live registration system but will provide students with the necessary Course Registration Numbers (CRNs) for manual entry. This proposal details the project's background, technical methodology, project timeline, and risk assessment.

2.0 BACKGROUND

The challenge of creating optimal university course timetables is a well-documented problem in the fields of operations research and artificial intelligence, formally known as the University Course Timetabling Problem (UCTP) [1]. The UCTP is recognized as an NP-hard problem, meaning that finding a perfect solution becomes computationally infeasible as the number of courses and constraints increases. Consequently, a variety of heuristic and algorithmic approaches have been developed to find good, practical solutions.

A common and effective method for solving such problems is to model them as a Constraint Satisfaction Problem (CSP) [2]. In a CSP model for course scheduling, the courses are variables, the available sections (lectures, labs, etc.) are the domains for those variables, and the constraints are rules such as "no two events can occupy the same time slot" and "all required components of a course must be scheduled." Algorithms such as backtracking, forward checking, and various local search heuristics are then applied to navigate the solution space and find valid timetables [2].

In practice, the state of the art can be divided into two categories of existing solutions: institutional schedulers and generic third-party tools.

1. **Institutional Schedulers:** Many universities have developed proprietary, in-house scheduling assistants. These tools are often integrated directly with the university's Student Information System (SIS). While effective, they are bespoke solutions tailored to a specific institution's data formats, academic rules, and IT infrastructure. As such, they are non-

transferable and do not offer a model that can be adapted for use at Carleton University without a significant institutional development effort.

2. **Generic Third-Party Tools:** Publicly available web applications (Coursicle, Free College Schedule Maker) provide a user-friendly interface for manually building timetables. However, their primary limitation is a lack of deep integration with institutional and student-specific data. They do not have access to a student's academic audit, cannot automatically validate prerequisites, and lack awareness of program-specific elective requirements. This forces the student back into the manual process of cross-referencing information from multiple sources.

At Carleton University, students currently rely on a combination of two disconnected tools: Carleton Central for registration and the public Carleton Course Scheduler for timetable visualization. Carleton Central serves as a transactional portal for enrollment but offers no strategic planning capabilities. The Carleton Course Scheduler, while useful for visualizing potential timetables, operates as a generic, third-party style tool. It requires students to manually input courses, cannot access a student's academic audit, and has no awareness of prerequisite completion or program-specific elective requirements. The responsibility for prerequisite validation, elective discovery, and conflict resolution thus remains entirely on the student. This fragmented and manual process is not only inefficient but also introduces a significant risk of error, which can delay graduation. This project addresses the clear gap between the capabilities of these existing tools and the specific, integrated needs of Carleton engineering students.

3.0 Project Objective

The primary objective of this project is to design, implement, and deploy a web-based intelligent course scheduling application for upper-year engineering students at Carleton University. The system aims to significantly reduce the time and complexity involved in academic planning by automating requirement validation and providing intelligent recommendations.

The specific functional and non-functional requirements that define the scope and quality of the final system are detailed below.

3.1 Table 1: Functional Requirements

ID	Requirement	Description
FR1	Academic Audit Parsing	The system shall parse an uploaded student academic audit (HTML or PDF) to extract a list of completed courses and identify program requirements.
FR2	Prerequisite Validation	The system shall validate that a student meets all prerequisite and co-requisite conditions for a selected course based on their audit or manual input.
FR3	Manual Course Entry	The system shall provide an interface for users to manually add or remove courses from their completed list and their desired schedule.
FR4	Elective Recommendation	The system shall display a list of eligible technical and complementary electives based on the user's program and remaining requirements.
FR5	Schedule Generation	The system shall generate all possible conflict-free timetables for a given set of desired courses, including all associated lab and tutorial sections.
FR6	Preference-based Ranking	The system shall rank generated schedules based on user-defined preferences (e.g., avoid morning classes, prefer compact days, keep a day free).

FR7	Interactive Timetable View	Generated schedules shall be displayed in a clear, interactive weekly calendar format.
FR8	Schedule Export	Users shall be able to export their final timetable, including Course Registration Numbers (CRNs), to a standard format (e.g., PDF, iCal).
FR9	Error and Conflict Feedback	The system shall provide clear, immediate feedback for scheduling conflicts, missing prerequisites, or other validation errors.

3.2 Table 2: Non-Functional Requirements

ID	Requirement	Description
NFR1	Performance	For a typical load of 4-6 courses, schedule generation and ranking must complete within 10 seconds. The web application must load in under 5 seconds.
NFR2	Usability	The user interface shall be intuitive, enabling a first-time user to generate a valid schedule with minimal instruction.
NFR3	Reliability	The system shall operate without crashing and handle invalid user inputs or failed audit parses gracefully.
NFR4	Compatibility	The application must be fully functional on the latest versions of major web browsers (Chrome, Firefox, Safari).
NFR5	Maintainability	The system's codebase will be modular and well-documented to facilitate future updates (e.g., updating course data for a new academic year).
NFR6	Security	All data transmission between the client and server will be encrypted using HTTPS.

3.3 Success Metrics

The success of the project will be evaluated against the following key metrics:

- **Validation Accuracy:** The prerequisite validation logic will be tested against a set of representative student scenarios. Success is defined as >95% accuracy in correctly identifying valid and invalid course selections.
- **Generation Completeness:** The scheduling engine must correctly generate all known valid timetables for a set of predefined course combinations.
- **Performance Benchmark:** The system must meet the performance target defined in NFR1 during user acceptance testing.
- **User Satisfaction:** Usability will be measured through a user testing phase with a small set of student volunteers. Success will be determined by a System Usability Scale (SUS) score of 70 or higher, indicating above-average usability.

4.0 Proposed System Architecture

This section outlines the high-level design of the Carleton Smart Scheduler, the chosen technology stack, and the expected user workflow. The architecture is designed to be modern, scalable, and maintainable.

4.1 High-Level Design

The system will be implemented as a full-stack application using Next.js. This architecture integrates the client-side user interface and the server-side business logic into a single, cohesive codebase. The design consists of three primary components:

1. **Client (Browser):** A dynamic front-end built with React and TypeScript. This component is responsible for rendering the user interface, capturing user input (such as audit file uploads, course selections, and preferences), and displaying the generated timetables. It will communicate with the backend via RESTful API calls.

2. **Backend (Next.js Server):** The server-side logic, running in a Node.js environment. This component will handle all core functionality, including:
    - Receiving and parsing academic audit files.
    - Validating course prerequisites against the database.
    - Executing the scheduling and ranking algorithms.
    - Interacting with the database via the Prisma ORM.

Database (PostgreSQL): A relational database that serves as the single source of truth for all course-related data. This includes course details, section times, prerequisite rules, and program requirements. The backend will communicate with the database exclusively through the Prisma client to ensure type-safe queries.

4.2 Technology Stack

The selection of technologies is based on ensuring type-safety, development efficiency, and robust performance. The following stack has been chosen for the project:

- **Web Framework: Next.js 14 (with App Router)**. We will use Next.js as our core framework. It is a full-stack React framework that provides an integrated solution for both frontend and backend development, simplifying our architecture and development workflow.
- **Language: TypeScript**. The entire application will be written in TypeScript. Its static typing ensures type-safety, reduces bugs, and improves code maintainability and team collaboration.
- **Database: PostgreSQL**. We will use PostgreSQL as our relational database. It is a powerful and reliable open-source system, ideal for storing the structured data of a course catalog, including prerequisites and section times.
- **ORM: Prisma**. To interact with the database, we will use Prisma. It is a modern, type-safe Object-Relational Mapper that simplifies database queries and integrates seamlessly with TypeScript, providing end-to-end type safety from the database to the client.
- **Audit Parsing Libraries**. To extract data from student audits, we will employ a dual strategy. Our **primary method** will be parsing user-provided HTML

files using a robust server-side library (Cheerio), as HTML offers a more reliable and structured format. As a **secondary objective**, we will investigate the feasibility of supporting PDF uploads by evaluating relevant libraries (e.g., pdf-parse) to handle the unstructured nature of PDF documents.

- **React UI Component Library**. The user interface will be built using a comprehensive React component library (e.g., Material-UI, Ant Design). This approach will accelerate UI development, ensure a consistent design, and provide built-in accessibility features.
- **Deployment Platform: Vercel**. The final application will be deployed on Vercel. As a cloud platform optimized for Next.js, it provides seamless continuous integration and deployment (CI/CD), allowing our team to focus on development rather than infrastructure management.

4.3 User Workflow

The system is designed for maximum flexibility, allowing users to explore schedules freely while providing intelligent guidance when requested.

1. **Course Selection:** The user begins by selecting their desired courses for the upcoming term. This is the primary and only required step to start building a schedule.
2. **Prerequisite Advising:** As courses are selected, the system will clearly display their prerequisites.
    - If an academic audit has **not** been uploaded, these prerequisites are shown for the user's information and self-assessment. The system will not block schedule generation.
    - If an academic audit **has** been uploaded, the system will automatically cross-reference the prerequisites against the user's completed courses and display a clear status icon (a green checkmark for "Met," a yellow warning for "Not Met"). This provides an immediate validation check.

3. **Enhance with Academic History:** To enable automated validation and recommendations, the user can provide their academic history at any time through one of two methods:
    - **Automated Audit Parsing:** Uploading an academic audit (HTML format preferred for reliability).
    - **Manual Entry:** Manually selecting completed courses from a master list.
4. **Elective Recommendation (If History Provided):** Once academic history is supplied, the system will feature a dedicated section suggesting eligible technical and complementary electives that are compatible with the user's core courses.
5. **Define Preferences (Optional):** The user can configure their scheduling preferences (time of day, schedule density) to influence the ranking of results.
6. **Generate and Review Schedules:** The user generates timetables. The system produces all conflict-free options. If preferences were set, the results are ranked. If an audit was provided, schedules containing courses with unmet prerequisites may be flagged but will still be displayed.

Finalize and Export: The user can browse the generated schedules, and export their final choice with the necessary Course Registration Numbers (CRNs).

5.0 Methodology

This section details the technical methods and processes that will be employed to develop the Carleton Smart Scheduler. The project will follow an iterative development model, allowing for continuous refinement of features based on testing and feedback.

5.1 Audit Parsing and Prerequisite Validation

The methodology for handling academic audits is designed to maximize reliability while exploring support for multiple formats.

- **Primary Method (HTML Parsing):** Our core strategy will focus on parsing academic audits saved as HTML files. A user will upload the file, which will be processed by a Next.js API route. On the server, a Node.js-based HTML parsing library will be used to traverse the Document Object Model (DOM) of the audit. We will identify and extract course codes ("SYSC 4907") and completion status by targeting specific HTML tags and CSS selectors. Regular expressions will be applied to standardize the extracted data. This method is prioritized due to the structured and predictable nature of HTML.
- **Secondary Investigation (PDF Parsing):** As a secondary objective, the team will investigate methods for parsing PDF audits. This is a higher-risk task due to the unstructured format of PDFs. The approach will involve using a library to extract the raw text content from the PDF file. Subsequently, a series of complex regular expressions and pattern-matching algorithms will be developed to identify course codes and grades from the text stream.
- **Validation Logic:** Once a list of completed courses is obtained (either from parsing or manual input), the prerequisite validation will be performed on the backend. When a user selects a course to add to their schedule, the system will query the PostgreSQL database via Prisma to retrieve that course's prerequisite rules. This list of required courses will then be checked against the user's list of completed courses to determine eligibility. The validation result will be sent to the client to provide immediate feedback.

5.2 Scheduling and Ranking Engine

The core of the application is the engine that generates and ranks timetables.

- **Scheduling Algorithm:** The problem of generating valid timetables will be modeled as a **Constraint Satisfaction Problem (CSP)**.
    - **Variables:** The set of courses the user wishes to take.
    - **Domains:** For each course (variable), the domain is the set of all its available sections (lectures, labs, and tutorials).
    - **Constraints:** The primary constraints are (1) **Time-Conflict Constraint**, ensuring no two selected sections overlap in time, and (2) **Completeness Constraint**, ensuring that a valid timetable includes one of each required course component (e.g., one lecture, one lab if required).
- To find all possible solutions that satisfy these constraints, we will research and implement an appropriate **constraint-solving search algorithm**. This approach will systematically explore valid combinations of course sections. The algorithm will be designed to be efficient by incorporating optimization techniques, such as pruning the search space (i.e., abandoning a partial schedule as soon as a conflict is detected), which is essential for meeting the system's performance requirements (NFR1).
- **Ranking Model:** After generating all valid, conflict-free schedules, a **heuristic scoring function** will be used for ranking. This function will evaluate each timetable against the user's selected preferences. For example, a schedule might receive positive points for having no classes on a Friday or negative points for long gaps between classes. The timetables will then be presented to the user, sorted from the highest to the lowest score.

5.3 Database Design

The system will be supported by a relational database schema in PostgreSQL.

- **Data Sourcing and Population:** The system's data will be sourced from two distinct channels:
    - **Timetable Data (Provided):** The data for course offerings, including sections, CRNs, times, and instructors for a given academic term, will be provided by the project supervisor in a structured format (e.g., JSON, CSV). This data will be ingested and used to populate the Sections table in our database.
    - **Prerequisite Data (Web Scraped):** Prerequisite rules are not included in the provided timetable data and must be sourced from the official Carleton University undergraduate calendar. We will develop a **one-time web scraping script** using a Node.js library (such as Puppeteer or Playwright) to systematically navigate the calendar website. The script will parse the HTML of each course page to extract the text describing its prerequisite and co-requisite rules. This text will be processed and structured to populate our Prerequisites relational table.
- **Schema Design:** Key entities will include:
    - Courses: Stores static information about each course (e.g., course code, title).
    - Sections: Stores the term-specific offering details provided by the supervisor.
    - Prerequisites: A relational table modeling the complex prerequisite rules obtained via web scraping.
- **Database Management:** The Prisma ORM will be used to manage the database schema through migrations and to provide a type-safe client for all database queries from the backend.

5.4 Security and Privacy Considerations

While the system requires access to sensitive academic information, the protection of student privacy is a critical consideration. Our methodology incorporates the principle of data minimization and ephemeral processing. Uploaded audit files will not be permanently stored on the server. They will be processed temporarily to extract the necessary course data and then discarded immediately. All data transmitted between the client and server will be encrypted using standard HTTPS protocols.

5.5 Testing Strategy

A multi-layered testing strategy will be implemented to ensure the system is robust, correct, and usable.

- **Unit Testing:** Individual functions, particularly for the parsing logic and scheduling algorithm, will be tested in isolation using a framework like Jest or Vitest.
- **Integration Testing:** We will test the interactions between major components, such as verifying that the frontend correctly communicates with the backend API and that the API correctly queries the database.

User Acceptance Testing (UAT): To validate usability and gather qualitative feedback, we will conduct testing sessions with a cohort of volunteer students. This will be a key part of evaluating our success metrics (Section 3.3).

6.0 Relation to Degree Program

As a team composed entirely of Software Engineering students, this project serves as a comprehensive application of the core competencies developed throughout our degree program. The project's lifecycle, from requirements analysis to deployment, directly maps to the principles of software design, development, and management central to the Software Engineering curriculum at Carleton University.

The project allows us to apply knowledge from several key domains of our program:

- **Software Design and Architecture:** Courses such as *SYSC 4120 (Software Architecture and Design)* have provided the foundation for architecting our system. We will apply principles like separation of concerns to structure the Next.js application, defining clear boundaries between the client, server, and data layers. The design of RESTful APIs for client-server communication draws directly from concepts learned in our software design projects.
- **Algorithms and Data Structures:** The development of the core scheduling engine is a direct application of knowledge from *SYSC 2100 (Algorithms and Data Structures)*. Modeling the problem as a Constraint Satisfaction Problem (CSP) and implementing an efficient search algorithm to find solutions requires a deep understanding of algorithmic complexity, graph theory, and optimization techniques.
- **Database Management:** The design and implementation of the PostgreSQL database will leverage principles from *COMP 3005 (Database Management Systems)*. This includes creating a normalized relational schema to ensure data integrity, writing efficient queries for prerequisite validation, and using an ORM like Prisma to manage data persistence.
- **Web Application Development:** The frontend development and user interface design build upon practical skills acquired in project-based courses like *SYSC 4806 (Software Engineering Lab)*. We will use these skills to build a responsive, accessible, and intuitive user interface with React, ensuring a high-quality user experience.
- **Software Engineering Process:** The overall management of the project, including requirements engineering, version control with Git, iterative development, and a structured testing strategy, reflects the software development lifecycle methodologies taught throughout the program. This project is a practical exercise in applying these processes to deliver a robust and reliable product.

In essence, this project is not just a demonstration of coding skills but a capstone exercise in applying a holistic software engineering discipline to solve a complex, real-world problem.

7.0 Collective Skills of the Group

As a team of senior Software Engineering students, we possess a complementary set of technical skills and practical experiences that cover the full software development lifecycle required for this project. Our collective expertise spans frontend and backend development, database management, and system integration, ensuring that all aspects of the Carleton Smart Scheduler can be designed, implemented, and tested to a high standard.

The specific contributions and relevant skills of each team member are outlined below:

- **Hakeem Khan (Frontend Lead & UI/UX):** Hakeem brings professional web development experience from his co-op terms at Environment Canada and CRC Ottawa. His expertise in building enterprise applications with **React, TypeScript, and JavaScript** is directly applicable to the development of the scheduler's user interface. He will lead the implementation of the frontend using a modern React component library, ensuring the application is responsive, accessible, and intuitive. His experience integrating RESTful APIs will be crucial for connecting the client-side interface to the backend.
- **Pietro Adamvoski (Full-Stack & System Integration):** Pietro has hands-on experience in full-stack development, having built applications using **JavaScript, PHP, and Spring Boot**. His understanding of the MVC (Model-View-Controller) architecture and experience implementing CRUD operations and RESTful APIs will be vital for integrating the project's various components. Pietro will focus on bridging the gap between the frontend and backend, developing API endpoints, and ensuring seamless data flow throughout the system.
- **Anas Ayoubi (Backend & Database Lead):** Anas will lead the backend development, focusing on the core logic of the application. His proficiency in **Java, Python, and SQL** makes him well-suited for designing the database schema and implementing the audit parsing logic. Anas will be responsible for developing the PostgreSQL database with Prisma, creating the data models,

and implementing the server-side validation and prerequisite-checking algorithms. His work will form the foundation of the system's "smart" capabilities.

- **Safiullah Rattar (Backend & Algorithm Development):** Safi will focus on the implementation of the core scheduling engine and the data scraping methodology. His skills in **TypeScript and Node.js** align perfectly with the chosen technology stack. He will be responsible for developing the web scraping scripts to populate our database with prerequisite data and will work closely with the team to implement the constraint-solving algorithm for schedule generation. His focus will be on ensuring the backend logic is efficient, scalable, and correctly implemented.

8.0 Project Timeline and Milestones

The project will be executed over the Fall 2025 and Winter 2026 academic terms, aligned with the key deliverables of the SYSC 4907 capstone project. The timeline is structured to ensure an iterative development process, with a functional prototype available early for testing and refinement.

8.1 Phased Timeline

Fall 2025: System Foundation and Core Functionality

The primary focus of the fall term is on project planning, architectural design, and the implementation of a minimum viable product (MVP). By the end of this term, we will have a functional application capable of generating conflict-free schedules from manually selected courses.

- **September - October:** Finalize project scope, submit proposal, design system architecture and database schema.
- **November:** Implement backend infrastructure, develop the data scraping scripts, and populate the database. Begin development of the core scheduling algorithm.

- **December:** Develop the basic frontend for course selection and timetable display. Integrate frontend with backend APIs. Prepare and submit the Progress Report.

Winter 2026: Advanced Features, Testing, and Finalization

The winter term will focus on implementing the "smart" features of the scheduler, conducting rigorous testing, and preparing the final deliverables.

- **January:** Implement the academic audit parsing feature and refine the prerequisite validation logic. Prepare for the oral presentation.
- **February:** Develop the elective recommendation engine and the preference-based ranking model. Begin drafting the final report.
- **March:** Conduct user acceptance testing (UAT) with student volunteers. Refine the UI/UX based on feedback. Finalize all features and prepare for the Poster Fair.
- **April:** Complete the final report and project video. Perform final testing, code cleanup, and deploy the production-ready application.

8.2 Work Breakdown Structure

The project is broken down into the following phases and key tasks. A distinction is made between design (engineering problem-solving) and implementation (execution) to emphasize the application of core engineering principles.

Phase 1: System Design & Planning (Complete by Nov 14, 2025)

- **T1: Requirements Engineering & System Architecture**
  - Description: Define and document system requirements, design the high-level software architecture, and select the technology stack. **(Design)**
  - Owner(s): All
  - Target Date: Oct 24, 2025
- **T2: Detailed UI/UX Prototyping**

  - Description: Create high-fidelity mockups and interactive prototypes of the user interface using a tool like Figma. This defines the user workflow and component structure. **(Design)**
  - Owner(s): Hakeem Khan
  - Target Date: Nov 7, 2025
- **T3: Database & Data Model Design**
  - Description: Design the normalized relational schema for PostgreSQL and define the data models in Prisma. **(Design)**
  - Owner(s): Anas Ayoubi
  - Target Date: Nov 7, 2025
- **T4: Core Algorithm & Parser Design**
  - Description: Research and design the constraint-solving algorithm and the HTML audit parsing strategy. **(Design)**
  - Owner(s): Safi Rattar, Pietro Adamvoski
  - Target Date: Nov 14, 2025

Phase 2: Backend Implementation & Frontend Scaffolding (Complete by Jan 17, 2026)

- **T5: Data Sourcing & Population**
  - Description: Implement the web scraping script for prerequisites and populate the database. **(Implementation)**
  - Owner(s): Safi Rattar
  - Target Date: Nov 28, 2025
- **T6: Frontend Project Setup & Component Scaffolding**
  - Description: Initialize the Next.js frontend, set up the UI library, and create placeholder components based on the T2 prototype, using mock data. **(Implementation)**
  - Owner(s): Hakeem Khan
  - Target Date: Dec 12, 2025
- **T7: Backend API & Database Implementation**
  - Description: Implement the Next.js API routes and core database logic based on the design. **(Implementation)**

  - Owner(s): Anas Ayoubi
  - Target Date: Dec 19, 2025
- **T8: Scheduling Engine & Parser Implementation**
  - Description: Implement the core scheduling algorithm and the HTML audit parser based on the design. **(Implementation)**
  - Owner(s): Safi Rattar, Pietro Adamvoski
  - Target Date: Jan 17, 2026

Phase 3: Integration & Advanced Logic (Complete by Feb 28, 2026)

- **T9: Prerequisite & Elective Logic Implementation**
  - Description: Implement the backend logic for validating prerequisites and recommending electives. **(Implementation)**
  - Owner(s): Anas Ayoubi
  - Target Date: Jan 31, 2026
- **T10: Preference-Based Ranking Model**
  - Description: Design and implement the heuristic scoring model for ranking schedules. **(Design & Implementation)**
  - Owner(s): Pietro Adamvoski
  - Target Date: Feb 14, 2026
- **T11: Full UI-Backend Integration**
  - Description: Connect the live frontend components to the backend API, replacing all mock data with real data to create a fully functional application. **(Implementation)**
  - Owner(s): Hakeem Khan, Pietro Adamvoski
  - Target Date: Feb 28, 2026

Phase 4: Testing & Refinement (Complete by Mar 28, 2026)

- **T12: System-Wide Testing**
  - Description: Develop and execute a comprehensive suite of unit, integration, and end-to-end tests for the entire system.
  - Owner(s): All
  - Target Date: Mar 14, 2026
- **T13: User Acceptance Testing (UAT) & Iteration**
  - Description: Conduct testing sessions with student volunteers, gather feedback, and implement necessary UI/UX refinements.
  - Owner(s): Hakeem Khan, Anas Ayoubi
  - Target Date: Mar 28, 2026

Phase 5: Finalization & Delivery (Complete by Apr 8, 2026)

- **T14: Final Deployment**
  - Description: Perform final code freeze, conduct regression testing, and deploy the application to the production environment on Vercel.
  - Owner(s): Safi Rattar, Pietro Adamvoski
  - Target Date: Apr 4, 2026
- **T15: Final Documentation & Deliverables**
  - Description: Complete the final report, project video, and presentation materials for submission.
  - Owner(s): All
  - Target Date: Apr 8, 2026

9.0 Risks and Mitigation Strategies

A thorough risk assessment is essential for successful project execution. We have identified potential risks in three key areas and have developed corresponding mitigation strategies to minimize their impact on the project's timeline and outcomes.

9.1 Technical Risks

- **Risk: Audit Parser Instability.** The structure of Carleton's academic audit HTML page is not guaranteed to be stable and may change without notice. Such a change could break our parser, disabling a core feature of the application. The complexity of PDF parsing presents an even higher risk of failure.
    - **Mitigation:** We will design the parser defensively, with robust error handling. If a parse fails, the system will degrade gracefully, notifying the user and seamlessly transitioning to the manual course entry mode. The application's core scheduling functionality will not depend on a successful parse. We will prioritize the more stable HTML format and treat PDF support as a secondary, higher-risk objective.
- **Risk: Algorithmic Performance.** The number of possible timetable combinations can grow exponentially with the number of selected courses and sections. A naive scheduling algorithm could lead to unacceptably long generation times, violating our performance requirements (NFR1).
    - **Mitigation:** We will benchmark the algorithm early in the development process using worst-case scenarios (e.g., 7 courses with many lab/tutorial options) to identify and address performance bottlenecks.

9.2 Data-Related Risks

- **Risk: Inaccuracy of Scraped Prerequisite Data.** The web scraping script for prerequisites relies on parsing unstructured text from the Carleton calendar. Complex or unusual prerequisite phrasings could be misinterpreted, leading to an inaccurate rule database.
    - **Mitigation:** After the initial scrape, the team will conduct a manual validation phase, cross-referencing a significant sample of the scraped rules against the official calendar to ensure accuracy. The scraper will

be designed with flexible pattern matching to handle common variations in language.

- **Risk: Stale Data.** The course catalog and timetable information change annually. If not updated, the application's data will become outdated and useless for future students.
    - **Mitigation:** The data sourcing scripts (for prerequisites and timetables) will be well-documented and designed to be re-runnable. The system's architecture will separate the application logic from the course data, allowing the database to be updated for a new academic year without requiring code changes to the core application.

9.3 Project Management Risks

- **Risk: Scope Creep.** The team may be tempted to add unplanned features (e.g., integration with the university system), which could divert resources from the core objectives and threaten deadlines.
    - **Mitigation:** We will strictly adhere to the functional requirements defined in Section 3.1. All proposed changes or additions will be subject to a formal review by the entire team to assess their impact on the schedule. The primary goal is to deliver a polished and robust version of the defined features.
- **Risk: Task Dependency Blockages.** The development of the frontend UI is dependent on the completion of the backend API. Delays in the backend could create a bottleneck for the entire project.

Mitigation: Our timeline is designed to mitigate this through parallel workstreams. The backend team will establish a clear API contract (defined data structures and endpoints) early in the process. The frontend team will then develop the UI components against this stable contract using mock data, allowing for concurrent development long before the backend logic is fully implemented.

10.0 Special Components and Facilities

This project is entirely software-based and does not require any specialized hardware, laboratory equipment, or physical facilities beyond standard development computers. All required software and services are open-source, available through free tiers, or provided by the university.

The key components required are:

- **Computing Resources:** Personal development laptops with access to a standard operating system (Windows, macOS, or Linux) are sufficient for all development tasks.
- **Software Stack:** The entire technology stack, including Node.js, Next.js, TypeScript, PostgreSQL, and Prisma, is open-source and freely available for development and deployment.
- **Data Sources:**
  - Access to the public Carleton University undergraduate calendar website is required for the prerequisite web scraping task.
  - The structured timetable data for course sections and times will be provided by the project supervisor.
- **Deployment Platform:** The project will be deployed on Vercel, which offers a comprehensive free tier that is sufficient for the development, testing, and hosting of this project.

No additional funding or special procurement is required for the successful completion of this project.

12.0 Conclusion

This project proposes the development of an intelligent course scheduler to address significant inefficiencies in the manual academic planning process for Carleton University's engineering students. By integrating academic audit parsing, prerequisite validation, and an elective recommendation engine, the system will provide a centralized, powerful tool that is currently lacking. The proposed architecture, methodology, and project plan are designed to produce a reliable, high-performance, and user-friendly application. The successful completion of this project will not only deliver a valuable tool to the student community but will also serve as a comprehensive demonstration of the software engineering principles and practices acquired throughout our degree program.

ACKNOWLEDGEMENT

We would like to acknowledge that this paper is the result of our own work and our supervisor, Prof. Jerome Talim, and we extend our gratitude to our supervisor for providing valuable feedback and guidance.

REFERENCES

[1] *I. Sbeity, M. Dbouk, and H. Kobeissi, "Combining the analytical hierarchy process and the genetic algorithm to solve the timetable problem," arXiv preprint, Sep. 2014.*

https://arxiv.org/abs/1409.2650

[2] *L. Zhang, Solving the timetabling problem using constraint satisfaction programming, MInfoSys thesis, University of Wollongong, Australia, 2005.*

https://ro.uow.edu.au/articles/thesis/Solving_the_timetabling_problem_using_constraint_satisfaction_programming/27827784

[3] *T. Schaub, "Answer set programming as a modeling language for course timetabling," Theory and Practice of Logic Programming, vol. 8, no. 5, pp. 541–564, Sep. 2008.*

https://www.cambridge.org/core/journals/theory-and-practice-of-logic-programming/article/answer-set-programming-as-a-modeling-language-for-course-timetabling/915CCB0FC4E2C1F1BE4DD71D766C793B

[4] *R. K. Thakur, N. K. Agrawal, and P. Kumar, "A practical approach to college timetable scheduling," Mathematical Modeling and Computing, vol. 11, no. 3, pp. 710–719, 2024.*

https://science.lpnu.ua/mmc/all-volumes-and-issues/volume-11-number-3-2024/practical-approach-college-timetable-scheduling

Last Page of Document